

Angular

Deep-Dive Interview Questions

29 Sections · 100+ Questions · Architecture, Scenarios & Cross-Questions
Covers Angular 2 → Angular 20+ · Standalone, Signals, Micro-Frontend

#	Section	#	Section
1	Components	16	Async Programming
2	App Startup Flow	17	Observables
3	Decorators – Basics	18	HttpClient
4	Data Binding	19	Async Pipe
5	Directives	20	RxJS Operators
6	Attribute Directives	21	Main Lifecycle Hooks
7	Pipes	22	Angular Forms – Template
8	Advance Decorators	23	Angular Forms – Reactive
9	Components Communication	24	JWT Authentication
10	Services & DI	25	Route Guards
11	Routing Basics	26	HTTP Interceptor
12	Observables Deep	27	Signals in Angular
13	HttpClient Deep	28	Angular 20+ Versions
14	Reactive Forms Deep	29	Standalone Architecture
15	JWT Deep	■■■	Micro-Frontend (Bonus)

SECTION 1 · COMPONENTS

Q1. What is an Angular Component and how does it differ from a Directive?

■ ANSWER

A Component is the most fundamental building block in Angular. It controls a portion of the screen called a view. Every Angular application has at least one component — the root component — that connects the component hierarchy with the page DOM.

Anatomy of a Component

- **@Component decorator** — metadata: selector, templateUrl, styleUrls, changeDetection strategy.
- **Template** — HTML describing how the view renders. Can be inline or external file.
- **Class** — TypeScript class containing the component's data, logic, and lifecycle hooks.
- **Styles** — encapsulated CSS that applies only to this component (ViewEncapsulation).

Component vs Directive

- A **Component** always has a template (view). A **Directive** never has its own template.
- A **Structural Directive** (e.g. *ngIf, *ngFor) reshapes the DOM by adding/removing elements.
- An **Attribute Directive** (e.g. ngClass, ngStyle) changes the appearance or behaviour of an existing element.
- You can have only **one** component per host element but **multiple** directives.

```
// Minimal component
@Component({
  selector: 'app-user-card',
  template: `
    <div class="card">
      <h2>{{ user.name }}</h2>
      <p>{{ user.email }}</p>
    </div>
  `,
  styles: [`.card { border: 1px solid #ccc; padding: 16px; }`]
})
export class UserCardComponent {
  @Input() user!: { name: string; email: string };
}
```

■ ChangeDetectionStrategy.OnPush makes Angular skip change detection for this component unless an @Input reference changes or an async event fires — a critical performance optimization.

■ CROSS / FOLLOW-UP QUESTIONS

- How does ViewEncapsulation work? What are the three modes (Emulated, None, ShadowDom)?
- What happens if you declare a component in two NgModules?
- How does Angular determine which component to render for a given selector?
- Can a component be used as a structural directive? Explain.
- How would you lazy-load a component without a route?

Q2. Explain the Angular component lifecycle hooks in order. When would you use each?

■ ANSWER

Angular creates, updates, and destroys components during its lifecycle. Lifecycle hooks are methods you implement on the component class to tap into these moments.

Hook Execution Order

- **ngOnChanges(changes: SimpleChanges)** — Fires when any `@Input` property changes. Called BEFORE `ngOnInit`. Receives a `SimpleChanges` object with previous and current values.
- **ngOnInit()** — Fires once after the first `ngOnChanges`. Use for one-time initialisation: API calls, subscriptions, complex setup. Do NOT use constructor for this.
- **ngDoCheck()** — Fires on every change detection cycle. Very expensive — use sparingly for custom dirty-checking.
- **ngAfterContentInit()** — Fires once after Angular projects external content (`ng-content`) into the component.
- **ngAfterContentChecked()** — Fires after every check of projected content.
- **ngAfterViewInit()** — Fires once after the component's view (and child views) are initialised. Safe to access `@ViewChild` here.
- **ngAfterViewChecked()** — Fires after every check of the component's view.
- **ngOnDestroy()** — Fires just before Angular destroys the component. Unsubscribe observables, clear timers, detach event handlers here.

```
export class MyComponent implements OnInit, OnDestroy {
  private sub!: Subscription;

  constructor(private svc: DataService) {}

  ngOnInit(): void {
    // Safe place for API calls
    this.sub = this.svc.getData().subscribe(data => this.data = data);
  }

  ngOnChanges(changes: SimpleChanges): void {
    if (changes['userId'] && !changes['userId'].firstChange) {
      // React to input change after first render
    }
  }
}
```

```

this.reload(changes['userId'].currentValue);
}
}

ngOnDestroy(): void {
  this.sub.unsubscribe(); // prevent memory leaks
}
}

```

■ Never call APIs in the constructor — Angular has not yet set `@Input` values. Use `ngOnInit` instead. Also avoid modifying state in `ngAfterViewChecked`; it triggers another change-detection cycle → `ExpressionChangedAfterItHasBeenCheckedError`.

■ CROSS / FOLLOW-UP QUESTIONS

- What is `ExpressionChangedAfterItHasBeenCheckedError` and how do you fix it?
- Difference between `ngOnInit` and a constructor in Angular?
- When would you use `ngDoCheck` over `ngOnChanges`?
- What happens to lifecycle hooks with `ChangeDetectionStrategy.OnPush`?
- How do you access a child component's public method from a parent — which hook is safe?

SECTION 2 · APPLICATION STARTUP FLOW

Q3. Walk me through exactly what happens when an Angular application bootstraps.

■ ANSWER

Step-by-step Startup Flow

- **1. index.html loads** — Browser downloads `index.html`. The `<app-root>` tag is present but empty.
- **2. main.ts executes** — The Angular CLI compilation output runs `main.ts` (or `main.js` after build).
- **3. platformBrowserDynamic() / bootstrapApplication()** — The platform is created. In module-based apps, `bootstrapModule(AppModule)` is called. In standalone apps, `bootstrapApplication(AppComponent, config)` is called.
- **4. AppModule / Providers are parsed** — Angular reads all providers, imports, declarations, and the bootstrap array.
- **5. Dependency Injection tree is built** — Root injector and platform injector are constructed.
- **6. Root component is instantiated** — Angular finds `<app-root>` in the DOM and mounts `AppComponent`.
- **7. First change-detection cycle runs** — Angular evaluates all bindings and renders the initial UI.
- **8. Router initialises** — If `RouterModule` is imported, the router reads the current URL and activates the matched route.

```
// Angular 17+ standalone bootstrap (main.ts)

import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app.component';
import { appConfig } from './app/app.config';

bootstrapApplication(AppComponent, appConfig)
  .catch(err => console.error(err));

// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient(),
    provideAnimations(),
  ],
};
```

■ In Angular 20+, the default project structure uses standalone components with no AppModule. The appConfig object replaces NgModule imports.

■ CROSS / FOLLOW-UP QUESTIONS

- What is the difference between platformBrowserDynamic and platformBrowser?
- How does Angular handle APP_INITIALIZER — when does it run?
- What happens if APP_INITIALIZER returns a rejected promise?
- How does the router interact with the bootstrap process?
- Explain the difference between root injector, platform injector, and environment injector.

SECTION 3 · DECORATORS – BASICS & ADVANCED

Q4. What are Angular Decorators? Explain all major types with examples.

■ ANSWER

Decorators are TypeScript design-pattern functions that attach metadata to classes, methods, properties, or parameters. Angular uses them heavily to configure its DI system and template engine.

Class Decorators

- **@Component** — Marks a class as a component and provides metadata (selector, template, styles, changeDetection).
- **@Directive** — Marks a class as a directive.
- **@Pipe** — Marks a class as a pipe (name, pure flag).

- **@NgModule** — Marks a class as an NgModule (declarations, imports, exports, providers, bootstrap).
- **@Injectable** — Marks a class as injectable via Angular's DI. `providedIn:'root'` creates a singleton.

Property / Parameter Decorators

- **@Input()** — Exposes a property to parent components for binding.
- **@Output()** — Exposes an EventEmitter so parents can listen to child events.
- **@ViewChild / @ViewChildren** — Gets a reference to a DOM element or child component inside the view.
- **@ContentChild / @ContentChildren** — Gets a reference to projected content (ng-content).
- **@HostListener** — Listens to events on the host element.
- **@HostBinding** — Binds a host element property to a component/directive property.
- **@Inject** — Manually specifies a DI token.

```
// Property decorator examples

@Component({ selector: 'app-demo', template: '' })
export class DemoComponent {
  @Input() title = ''; // Parent → Child
  @Output() clicked = new EventEmitter<string>(); // Child → Parent

  @ViewChild('myInput') inputRef!: ElementRef; // DOM ref

  @HostListener('mouseenter')
  onHover() { console.log('hovered'); }

  @HostBinding('class.active') isActive = false;
}
```

■ In Angular 14+ you can use the signal-based `input()` and `output()` functions instead of decorators in standalone components for better type inference and reactivity.

■ CROSS / FOLLOW-UP QUESTIONS

- What is the difference between `@ViewChild` and `@ContentChild`?
- When would you use `@HostBinding` vs `[attr.xxx]` template binding?
- What happens if you put `@Injectable` on a class but don't provide it anywhere?
- Explain the difference between `@Input()` and `input()` signal — Angular 17+.
- Can decorators be applied to arrow functions? Why or why not?

SECTION 4 · DATA BINDING

Q5. Explain all types of Data Binding in Angular with real examples.

■ ANSWER

Data binding is the mechanism that synchronises data between the component class and the template. Angular supports four main types:

1. Interpolation {{ expression }}

One-way: Component → Template. Evaluates an expression and renders it as a string. Used for displaying text.

```
<h1>Welcome, {{ user.name }}</h1>

<p>Total: {{ items.length * price | currency }}</p>
```

2. Property Binding [property]="expression"

One-way: Component → Template. Binds a component property to a DOM property (not HTML attribute).

```
<img [src]="user.avatarUrl" [alt]="user.name">

<button [disabled]="!form.valid">Submit</button>

<app-card [title]="pageTitle"></app-card>
```

3. Event Binding (event)="handler(\$event)"

One-way: Template → Component. Listens to DOM events and calls a component method.

```
<button (click)="onSave()">Save</button>

<input (input)="onSearch($event)" (keyup.enter)="submit()">

<app-cart (itemRemoved)="handleRemove($event)"></app-cart>
```

4. Two-Way Binding [(ngModel)]

Both directions simultaneously. Requires FormsModule. Under the hood it is [ngModel] + (ngModelChange).

```
<!-- Requires FormsModule -->

<input [(ngModel)]="username" placeholder="Enter name">

<!-- Equivalent manual two-way binding -->

<input [value]="username" (input)="username = $event.target.value">
```

■ Property binding [src] binds to the DOM property, while attribute binding [attr.colspan] binds to the HTML attribute. For ARIA and SVG, always use attr.xxx.

■ CROSS / FOLLOW-UP QUESTIONS

- What is the difference between [property] binding and attribute binding [attr.xxx]?
- Why does Angular have both interpolation and property binding? When to prefer which?
- How does two-way binding work internally — what is the banana-in-a-box syntax doing?
- Can you create custom two-way binding on your own component? Show the pattern.
- What are the security implications of using innerHTML binding vs interpolation?

SECTION 5 · DIRECTIVES

Q6. What is the difference between Structural and Attribute Directives? How do you create a custom one?

■ ANSWER

Directives are classes that add additional behaviour to elements in Angular applications.

Structural Directives

- Change the DOM layout by adding, removing, or replacing elements.
- Always prefixed with * in templates (syntactic sugar for <ng-template>).
- Examples: *ngIf, *ngFor, *ngSwitch, *ngTemplateOutlet.
- *ngIf='condition' expands to: <ng-template [ngIf]='condition'></ng-template>

Custom Structural Directive — appUnless

```
@Directive({ selector: '[appUnless]', standalone: true })
export class UnlessDirective {
  private hasView = false;

  constructor(
    private templateRef: TemplateRef<any>,
    private viewContainer: ViewContainerRef
  ) {}

  @Input() set appUnless(condition: boolean) {
    if (!condition && !this.hasView) {
      this.viewContainer.createEmbeddedView(this.templateRef);
      this.hasView = true;
    } else if (condition && this.hasView) {
      this.viewContainer.clear();
      this.hasView = false;
    }
  }
}

// Usage:
// <p *appUnless="isLoggedIn">Please log in.</p>
```

Custom Attribute Directive — appHighlight

```
@Directive({
```



```

selector: '[appHighlight]',
standalone: true
})
export class HighlightDirective {
  @Input() appHighlight = 'yellow';

  constructor(private el: ElementRef, private renderer: Renderer2) {}

  @HostListener('mouseenter')
  onEnter() {
    this.renderer.setStyle(
      this.el.nativeElement, 'backgroundColor', this.appHighlight
    );
  }

  @HostListener('mouseleave')
  onLeave() {
    this.renderer.removeStyle(this.el.nativeElement, 'backgroundColor');
  }
}

// Usage: <p appHighlight="lightblue">Hover me</p>

```

■ Always use `Renderer2` instead of directly manipulating `nativeElement`. This keeps your code compatible with server-side rendering (SSR/Universal) and web workers.

■ CROSS / FOLLOW-UP QUESTIONS

- Why can't you have two structural directives on the same element?
- How does Angular expand `*ngFor` internally into `ng-template`?
- What is the difference between `ElementRef` and `Renderer2`?
- How would you pass a context variable in a custom structural directive?
- When should you use a directive vs a component?

SECTION 6 · ATTRIBUTE DIRECTIVES: NGCLASS, NGSTYLE, NGSWITCH

Q7. Explain ngClass, ngStyle, and ngSwitch with practical examples and their internal mechanics.

■ ANSWER

ngClass — Dynamic CSS Classes

ngClass can accept a string, array, or object. The object form is most powerful: keys are class names, values are boolean expressions.

```
<!-- Object form (most common) -->
<div [ngClass]="{
  'active': isActive,
  'disabled': !isEnabled,
  'highlight': score > 90
}">Dashboard</div>

<!-- Array form -->
<div [ngClass]="['base-class', conditionalClass]"></div>

<!-- String form -->
<div [ngClass]="isAdmin ? 'admin-panel' : 'user-panel'"></div>
```

ngStyle — Dynamic Inline Styles

Applies CSS styles as key-value pairs. Values can be expressions. Useful for dynamic pixel values, colours, etc.

```
<div [ngStyle]="{
  'color': textColor,
  'font-size': fontSize + 'px',
  'opacity': isVisible ? 1 : 0
}">Styled Content</div>
```

ngSwitch — Template Switching

ngSwitch is a combination of structural directives. It is more performant than multiple *ngIf when switching between several exclusive views.

```
<div [ngSwitch]="role">
  <app-admin-panel *ngSwitchCase="'admin'"> </app-admin-panel>
  <app-manager-panel *ngSwitchCase="'manager'"> </app-manager-panel>
  <app-user-panel *ngSwitchCase="'user'"> </app-user-panel>
  <p *ngSwitchDefault>Unknown role</p>
</div>
```

■ ngSwitch uses identity comparison (===). For object comparison you may need a custom pipe or a computed string key.

■ CROSS / FOLLOW-UP QUESTIONS

- What is the performance difference between `*ngIf`/`*ngIf` chains vs `ngSwitch`?
- Can you use `ngClass` and `[class.xxx]` on the same element? Which takes priority?
- How does Angular's renderer apply `ngStyle` without triggering layout thrashing?
- How would you animate class changes applied via `ngClass`?

SECTION 7 · PIPES

Q8. What are Angular Pipes? Explain built-in pipes and how to create a pure vs impure custom pipe.

■ ANSWER

Pipes transform displayed values within a template. They don't mutate data — they return a new value for display. Pipes follow the `|` operator syntax.

Built-in Pipes

- **DatePipe**: `{{ date | date:'dd/MM/yyyy' }}`
- **CurrencyPipe**: `{{ price | currency:'INR':'symbol' }}`
- **DecimalPipe**: `{{ pi | number:'1.2-5' }}`
- **PercentPipe**: `{{ ratio | percent:'1.1-2' }}`
- **UpperCase / LowerCase / TitleCase**: `{{ name | titlecase }}`
- **SlicePipe**: `{{ items | slice:0:5 }}`
- **JsonPipe**: `{{ obj | json }}` — great for debugging
- **AsyncPipe**: `{{ obs$ | async }}` — subscribes/unsubscribes automatically
- **KeyValuePipe**: `*ngFor="let kv of obj | keyvalue"`

Custom Pure Pipe

```
@Pipe({ name: 'truncate', pure: true, standalone: true })
export class TruncatePipe implements PipeTransform {
  transform(value: string, limit = 100, trail = '...'): string {
    if (!value) return '';
    return value.length > limit
      ? value.substring(0, limit) + trail
      : value;
  }
}
```

// Usage: `{{ longText | truncate:50:'...' }}`

Custom Impure Pipe

An impure pipe (pure: false) re-evaluates on every change-detection cycle. Use for pipes that depend on mutable state (e.g., filtering a live array). Use sparingly — it's expensive.

```
@Pipe({ name: 'filterByStatus', pure: false, standalone: true })
export class FilterByStatusPipe implements PipeTransform {
  transform(items: Task[], status: string): Task[] {
    return items.filter(t => t.status === status);
  }
}
```

■ Prefer pure pipes wherever possible. For filtering/sorting large lists, consider doing it in the component class and exposing a filtered property, or using the async pipe with an observable pipeline.

■ CROSS / FOLLOW-UP QUESTIONS

- Why is the built-in async pipe considered superior to manual subscribe/unsubscribe?
- What is the difference between a pure and impure pipe at the change-detection level?
- Can you chain multiple pipes? Is there a performance concern?
- How would you create a pipe that handles null/undefined gracefully?
- Why should you NOT use impure pipes for filtering/sorting large datasets?

SECTION 8 · ADVANCE DECORATORS & MODULES

Q9. Explain NgModule in depth — declarations, imports, exports, providers, and bootstrap.

■ ANSWER

NgModule is a class decorated with @NgModule that acts as a cohesion boundary for a set of related components, directives, and pipes. In Angular 20+, NgModules are optional (standalone architecture), but understanding them is essential for legacy codebases.

NgModule Metadata Fields

- **declarations** — Components, directives, and pipes that BELONG to this module. They are only available within this module unless exported.
- **imports** — Other NgModules whose exported declarations are available in this module's templates.
- **exports** — Subset of declarations (and imported modules) that should be visible to importing modules.
- **providers** — Services registered at the module injector level. Avoid module-level providers; prefer providedIn:'root'.
- **bootstrap** — Root component(s) that Angular inserts into index.html. Only used in the root AppModule.

```
@NgModule({
  declarations: [
```

```

    UserCardComponent,
    EmailPipe,
    HighlightDirective,
  ],
  imports: [
    CommonModule,
    ReactiveFormsModule,
    RouterModule.forChild(userRoutes),
  ],
  exports: [
    UserCardComponent, // Make available to other modules
    EmailPipe,
  ],
  providers: [
    UserService, // Module-scoped (avoid unless needed)
  ]
})

export class UserModule {}

```

Feature Module vs Shared Module vs Core Module

- **Feature Module** — Contains everything for one feature (lazy-loaded). E.g., UserModule, OrderModule.
- **Shared Module** — Contains reusable components/pipes/directives imported by multiple feature modules. Never has providers.
- **Core Module** — Singleton services, interceptors, guards. Imported ONLY in AppModule. Has a guard to prevent re-import.

■ In Angular 17+, the recommended approach is standalone components + provideRouter() + provideHttpClient() in app.config.ts, eliminating the need for NgModules entirely.

■ CROSS / FOLLOW-UP QUESTIONS

- What happens if you declare a component in two NgModules?
- Why should SharedModule not have providers?
- What is the forRoot() / forChild() pattern and why is it used for RouterModule?
- How do you prevent a CoreModule from being imported more than once?
- What replaces NgModules in standalone Angular? What are the migration steps?

SECTION 9 · COMPONENTS COMMUNICATION

Q10. Explain all the ways components can communicate in Angular. Cover parent-child, sibling, and service-based patterns.

■ ANSWER

1. Parent → Child: @Input()

```
// child.component.ts
@Input() userName!: string;
@Input({ required: true }) userId!: number; // Angular 16+

// parent.template.html
<app-child [userName]="currentUser.name" [userId]="currentUser.id">
</app-child>
```

2. Child → Parent: @Output() + EventEmitter

```
// child.component.ts
@Output() itemSelected = new EventEmitter<Product>();

onSelect(p: Product) {
  this.itemSelected.emit(p);
}

// parent.template.html
<app-child (itemSelected)="handleSelection($event)"></app-child>
```

3. Parent → Child: @ViewChild (direct reference)

```
// parent.component.ts
@ViewChild(TimerComponent) timer!: TimerComponent;

ngAfterViewInit() {
  this.timer.start(); // Call child method directly
}
```

4. Sibling / Any Component: Shared Service with Subject/BehaviorSubject

```
@Injectable({ providedIn: 'root' })
export class CartService {
  private cartItems$ = new BehaviorSubject<CartItem[]>([]);
  readonly cart$ = this.cartItems$.asObservable();

  addItem(item: CartItem) {
    const current = this.cartItems$.getValue();
    this.cartItems$.next([...current, item]);
  }
}
```

```
}  
}  
  
// In any component:  
cart$ = this.cartService.cart$;
```

5. Angular Signals (Angular 16+)

```
// shared.store.ts  
export const count = signal(0);  
export const doubled = computed(() => count() * 2);  
  
// component A  
count.set(5);  
  
// component B (auto-reactive)  
template: `

{{ doubled() }}

`
```

■ **BehaviorSubject vs Subject:** BehaviorSubject always emits the last value to new subscribers. Use it for state that new components need immediately (e.g., auth state, cart). Use Subject for events.

■ CROSS / FOLLOW-UP QUESTIONS

- What is the difference between BehaviorSubject, Subject, and ReplaySubject?
- When would you use a service for communication vs @Input/@Output?
- How do signals compare to BehaviorSubject for state management?
- What is the ng-content / Content Projection pattern?
- How do you communicate between components in different lazy-loaded modules?
- What is the difference between a smart (container) component and a dumb (presentational) component?

SECTION 10 · SERVICES & DEPENDENCY INJECTION

Q11. Explain Angular's Dependency Injection system in depth — injectors, tokens, hierarchical DI.

■ ANSWER

Angular's DI system is a design pattern where a class requests dependencies instead of creating them. Angular's injector creates and delivers those dependencies.

Injector Hierarchy

- **NullInjector** — Top of the tree. Always throws unless you use @Optional().
- **EnvironmentInjector (Root)** — providedIn:'root' services live here. Singleton across the entire app.

- **ModuleInjector** — Providers in `NgModule.providers` or lazy-loaded module.
- **ElementInjector** — Providers in `@Component.providers` — one instance per component instance.

Providing Services

```
// Singleton (most common)

@Injectable({ providedIn: 'root' })
export class AuthService { }

// Component-scoped (new instance per component)
@Component({
  providers: [DialogService] // fresh instance for this component tree
})
export class ModalComponent { }

// Custom token
const API_URL = new InjectionToken<string>('API_URL');

// In providers:
{ provide: API_URL, useValue: 'https://api.example.com' }

// In constructor:
constructor(@Inject(API_URL) private url: string) {}
```

useClass, useValue, useFactory, useExisting

```
providers: [
  // Swap implementation
  { provide: LoggerService, useClass: ProductionLoggerService },

  // Provide a primitive
  { provide: MAX_RETRIES, useValue: 3 },

  // Factory with dependencies
  {
    provide: HttpClient,
    useFactory: (backend: HttpBackend) => new HttpClient(backend),
    deps: [HttpBackend]
  },

  // Alias – both tokens point to same instance
```



```
{ provide: AbstractUserService, useExisting: UserService }  
]
```

■ The lookup walks UP the injector tree. A child injector shadows a parent's provider. This is how you can have different service instances for different component sub-trees.

■ CROSS / FOLLOW-UP QUESTIONS

- ▶ What is the difference between `providedIn:'root'` and `providedIn:'any'`?
- ▶ How would you provide a different implementation of a service in testing?
- ▶ What is the `@Optional()` decorator and when do you use it?
- ▶ Explain the `useExisting` vs `useClass` difference with a real-world scenario.
- ▶ How does lazy-loading affect the injector hierarchy?
- ▶ What is tree-shakeable provider and why does `providedIn:'root'` enable tree-shaking?

SECTION 11 · ROUTING BASICS

Q12. How does Angular routing work? Cover router configuration, lazy loading, route parameters, and child routes.

■ ANSWER

Router Configuration

```
// app.routes.ts  
  
export const routes: Routes = [  
  { path: '', redirectTo: 'home', pathMatch: 'full' },  
  { path: 'home', component: HomeComponent },  
  
  // Route with parameter  
  { path: 'user/:id', component: UserDetailComponent },  
  
  // Child routes  
  {  
    path: 'dashboard',  
    component: DashboardComponent,  
    children: [  
      { path: 'analytics', component: AnalyticsComponent },  
      { path: 'reports', component: ReportsComponent },  
    ],  
  },  
],
```

```
// Lazy-loaded feature module (Angular 17+ standalone)
{
  path: 'admin',
  loadChildren: () =>
    import('./admin/admin.routes').then(m => m.adminRoutes),
  canActivate: [AuthGuard],
},

{ path: '**', component: NotFoundComponent }
];
```

Reading Route Parameters

```
// Snapshot (does NOT update if same component reused)
const id = this.route.snapshot.paramMap.get('id');

// Observable (updates on param change – PREFERRED)
this.route.paramMap.subscribe(params => {
  this.userId = +params.get('id')!;
  this.loadUser(this.userId);
});

// Query params
this.route.queryParamMap.subscribe(qp => {
  this.page = +qp.get('page')! || 1;
});
```

Navigating Programmatically

```
// Absolute navigation
this.router.navigate(['/user', userId]);

// Relative navigation
this.router.navigate(['../sibling'], { relativeTo: this.route });

// With query params
this.router.navigate(['/search'], {
  queryParams: { q: 'angular', page: 1 },
  queryParamsHandling: 'merge' // preserve existing
});
```

■ Always use `loadChildren` (lazy loading) for feature routes. This reduces the initial bundle size and improves Time to Interactive (TTI) — a key Lighthouse metric.

■ CROSS / FOLLOW-UP QUESTIONS

- What is the difference between `pathMatch:'full'` and `pathMatch:'prefix'`?
- How does Angular's router preload strategies work (`PreloadAllModules` vs `NoPreloading` vs custom)?
- What is `RouterLink` vs `Router.navigate` — when to use which?
- Explain route resolvers. How do they differ from data fetching in `ngOnInit`?
- What is a secondary (named) router outlet?
- How would you pass complex data between routes without query params?

SECTION 12 · OBSERVABLES & RXJS

Q13. Explain Observable, Observer, Subscriber, and the data flow. How does RxJS relate to Angular?

■ ANSWER

An Observable is a lazy push-based data stream. It emits values over time. Unlike Promises, Observables are lazy (nothing happens until subscribe), cancellable, and can emit multiple values.

Core Concepts

- **Observable** — The producer. Defines what values to emit and when.
- **Observer** — The consumer. An object with `next()`, `error()`, `complete()` callbacks.
- **Subscription** — The result of `subscribe()`. Holds a reference you can call `.unsubscribe()` on.
- **Operators** — Pure functions (`map`, `filter`, `switchMap`...) that transform the stream.
- **Subject** — Both Observable and Observer. **Multicast** — multiple subscribers share one execution.

```
// Creating an Observable from scratch
const obs$ = new Observable<number>(subscriber => {
  subscriber.next(1);
  subscriber.next(2);
  subscriber.next(3);
  subscriber.complete();

  // Teardown logic (runs on unsubscribe)
  return () => console.log('Cleaned up');
});

// Subscribing
const sub = obs$.subscribe({
```

```

next: v => console.log('Got:', v),
error: err => console.error('Error:', err),
complete: () => console.log('Done'),
});

// Later – clean up
sub.unsubscribe();

```

Observable vs Promise

- **Lazy vs Eager:** Observable only executes when subscribed. Promise executes immediately.
- **Multiple values:** Observable emits many values over time. Promise resolves once.
- **Cancellable:** You can unsubscribe from an Observable. You cannot cancel a Promise.
- **Operators:** Observables have a rich operator library (RxJS). Promises have `.then()/.catch()` only.

■ Angular uses Observables throughout: HttpClient returns Observables, the Router exposes Observables, reactive forms have `valueChanges` Observable, and the async pipe consumes Observables.

■ CROSS / FOLLOW-UP QUESTIONS

- What is a cold vs hot Observable? Give an Angular example of each.
- What happens if you don't unsubscribe from an Observable? What is the memory leak?
- When should you use `takeUntilDestroyed()` vs manual unsubscribe?
- What is the difference between `Observable.of()` and `Observable.from()`?
- Explain multicasting — `share()`, `shareReplay()`, `publish()`.

Q14. Explain the key RxJS operators: map, filter, switchMap, mergeMap, concatMap, exhaustMap, tap, catchError, forkJoin, combineLatest.

■ ANSWER

Transformation Operators

```

// map – transform each value
users$.pipe(
  map(users => users.filter(u => u.active))
)

// switchMap – cancel previous inner obs, use latest (search, autocomplete)
searchInput.valueChanges.pipe(
  debounceTime(300),
  switchMap(query => this.api.search(query))
)

// mergeMap – concurrent – all inner obs run in parallel (file uploads)

```

```

files$.pipe(
  mergeMap(file => this.uploadService.upload(file))
)

// concatMap – sequential – waits for each inner obs to complete (ordered ops)
actions$.pipe(
  concatMap(action => this.api.process(action))
)

// exhaustMap – ignore new emissions while inner obs is active (login button)
loginClick$.pipe(
  exhaustMap(() => this.authService.login(credentials))
)

```

Utility & Error Operators

```

// tap – side effects without changing stream
obs$.pipe(
  tap(v => console.log('Before map:', v)),
  map(v => v * 2),
  tap(v => console.log('After map:', v))
)

// catchError – recover from error
this.api.getUser(id).pipe(
  catchError(err => {
    this.notify.error(err.message);
    return of(null); // fallback value
  })
)

// forkJoin – parallel, emits when ALL complete (like Promise.all)
forkJoin({
  user: this.api.getUser(id),
  orders: this.api.getOrders(id),
  address: this.api.getAddress(id),
}).subscribe(({ user, orders, address }) => { ... })

// combineLatest – emits whenever ANY source emits (dashboard live data)

```

```
combineLatest([filter$, sort$, page$]).pipe(
  switchMap(([filter, sort, page]) => this.api.list(filter, sort, page))
)
```

■ The most common interview trap: use `switchMap` for user searches (cancel stale requests), NOT `mergeMap` (which would process all results including outdated ones).

■ CROSS / FOLLOW-UP QUESTIONS

- When would you choose `mergeMap` over `switchMap`? Give a real scenario.
- What is the difference between `forkJoin` and `combineLatest`?
- How do you retry a failed HTTP request with exponential backoff using RxJS?
- What does `shareReplay(1)` do and when is it useful?
- Explain the difference between `throttleTime` and `debounceTime`.

SECTION 13 · HTTPCLIENT

Q15. How does Angular's HttpClient work? Explain request methods, error handling, and the request/response pipeline.

■ ANSWER

Making HTTP Requests

```
@Injectable({ providedIn: 'root' })
export class UserService {
  private http = inject(HttpClient);
  private base = 'https://api.example.com';

  // GET – typed response
  getUser(id: number): Observable<User> {
    return this.http.get<User>(`${this.base}/users/${id}`);
  }

  // POST with headers
  createUser(user: CreateUserDto): Observable<User> {
    const headers = new HttpHeaders({ 'Content-Type': 'application/json' });
    return this.http.post<User>(`${this.base}/users`, user, { headers });
  }

  // PUT / PATCH / DELETE
```

```

updateUser(id: number, dto: Partial<User>): Observable<User> {
  return this.http.patch<User>(`${this.base}/users/${id}`, dto);
}

deleteUser(id: number): Observable<void> {
  return this.http.delete<void>(`${this.base}/users/${id}`);
}

// Query params
search(query: string, page: number): Observable<PageResult<User>> {
  const params = new HttpParams()
    .set('q', query)
    .set('page', page.toString());
  return this.http.get<PageResult<User>>(`${this.base}/users`, { params });
}
}

```

Centralised Error Handling

```

// http-error.interceptor.ts
export const httpErrorInterceptor: HttpInterceptorFn = (req, next) => {
  return next(req).pipe(
    catchError((err: HttpErrorResponse) => {
      if (err.status === 401) {
        inject(AuthService).logout();
        inject(Router).navigate(['/login']);
      } else if (err.status === 0) {
        console.error('Network error – no connectivity');
      }
      return throwError(() => err);
    })
  );
};

```

■ In Angular 15+, use functional interceptors (`HttpInterceptorFn`) registered via `withInterceptors([...])` in `provideHttpClient()`. Class-based interceptors still work but are being deprecated.

■ CROSS / FOLLOW-UP QUESTIONS

- How do you add auth headers to every request using an interceptor?
- What is `HttpContext` and how do you skip an interceptor for specific requests?
- How would you implement request caching with an HTTP interceptor?
- Explain the difference between `HttpParams` (immutable) and a plain object for query params.

► How would you display a global loading spinner for all HTTP requests?

SECTION 14 · ASYNC PIPE

Q16. Why is the async pipe preferred over manual subscribe/unsubscribe?

■ ANSWER

The async pipe subscribes to an Observable or Promise and returns the latest emitted value. Angular automatically unsubscribes when the component is destroyed.

Manual subscribe (error-prone)

```
// BAD – risk of memory leak if ngOnDestroy is missed

users: User[] = [];

private sub!: Subscription;

ngOnInit() {
  this.sub = this.userService getUsers().subscribe(u => this.users = u);
}

ngOnDestroy() { this.sub.unsubscribe(); } // easy to forget
```

Async pipe (recommended)

```
// GOOD – automatic subscription management

readonly users$ = this.userService getUsers();

// template

<ng-container *ngIf="users$ | async as users; else loading">
  <app-user-card *ngFor="let u of users" [user]="u"></app-user-card>
</ng-container>

<ng-template #loading><app-spinner></app-spinner></ng-template>
```

Multiple async subscriptions — share to avoid duplicate calls

```
// Without shareReplay → 3 HTTP calls

users$ = this.api.getUsers();

<div *ngIf="users$ | async as u">{{ u.length }}</div>

<ul><li *ngFor="let u of users$ | async">{{ u.name }}</li></ul>

// WITH shareReplay(1) → 1 HTTP call, multicasted

users$ = this.api.getUsers().pipe(shareReplay(1));
```


■ The async pipe also triggers change detection when a new value arrives, making it work correctly with OnPush change detection — a major advantage over manual subscriptions.

■ CROSS / FOLLOW-UP QUESTIONS

- How does the async pipe interact with ChangeDetectionStrategy.OnPush?
- What are the drawbacks of the async pipe?
- How do you combine multiple observables in a template using async pipe?
- What is the let syntax in *ngIf with async pipe?

SECTION 15 · MAIN LIFECYCLE HOOKS — DEEP DIVE

Q17. Scenario: You have an OnPush component receiving an @Input array. Items are added to the array externally. Why doesn't the component update, and how do you fix it?

■ ANSWER

This is one of the most common OnPush pitfalls in Angular interviews.

Root Cause

With ChangeDetectionStrategy.OnPush, Angular only runs change detection for a component when:

- An @Input reference changes (different object reference, not just mutation).
- An event originated from the component or its children.
- An async pipe emits a new value.
- Change detection is triggered manually via ChangeDetectorRef.

```
// PROBLEM – mutating the array doesn't change its reference
export class ParentComponent {
  items = ['a', 'b', 'c'];

  addItem() {
    this.items.push('d'); // ■ same reference – OnPush child won't update
  }
}

// FIX 1 – create new array (immutable update)
addItem() {
  this.items = [...this.items, 'd']; // ■ new reference
}
```

```
// FIX 2 – use ChangeDetectorRef in the child
@Component({ changeDetection: ChangeDetectionStrategy.OnPush })
export class ListComponent {
  constructor(private cdr: ChangeDetectorRef) {}

  forceUpdate() {
    this.cdr.markForCheck(); // Schedule check at next cycle
    // OR
    this.cdr.detectChanges(); // Immediately run CD for this subtree
  }
}

// FIX 3 – use Observable + async pipe (best for OnPush)
items$ = new BehaviorSubject(['a', 'b', 'c']);

addItem() {
  this.items$.next([...this.items$.getValue(), 'd']);
}
```

■ `markForCheck()` marks the component and its ancestors dirty for the next CD cycle. `detectChanges()` runs synchronously. In most cases, prefer immutable updates + async pipe.

■ CROSS / FOLLOW-UP QUESTIONS

- What is the difference between `markForCheck()` and `detectChanges()`?
- How does `NgZone` relate to change detection?
- What is `zone.js` and can Angular run without it?
- How do you detach a component from change detection for performance?
- Explain the difference between Default and OnPush change detection strategies in a tree of 100 components.

SECTION 16 · ANGULAR FORMS — TEMPLATE DRIVEN & REACTIVE

Q18. Compare Template Driven Forms vs Reactive Forms. When do you choose each?

■ ANSWER

Template Driven Forms

- Defined in the HTML template using `ngModel`, `ngForm`, `ngModelGroup`.
- Form structure is implicit — Angular creates `FormControl` instances automatically.
- Two-way binding makes it easy for simple forms.
- Requires `FormsModule`.
- Validation via HTML attributes (`required`, `minlength`) or custom directives.
- Less testable — logic is in the template.

```
<form #loginForm="ngForm" (ngSubmit)="onSubmit(loginForm)">
<input
  name="email"
  ngModel
  required
  email
  #email="ngModel">
<span *ngIf="email.invalid && email.touched">
  Enter a valid email
</span>
<button [disabled]="loginForm.invalid">Login</button>
</form>
```

Reactive Forms (Model-Driven)

- Form structure defined in the component class using `FormControl`, `FormGroup`, `FormArray`.
- Explicit, immutable, and predictable. Easy to unit test.
- Requires `ReactiveFormsModule`.
- Validation via `Validators` class or custom `ValidatorFn`.
- Supports dynamic forms, complex cross-field validation.
- `valueChanges` and `statusChanges` are `Observables` — integrate with `RxJS`.

```
this.form = this.fb.group({
  email: ['', [Validators.required, Validators.email]],
  password: ['', [Validators.required, Validators.minLength(8)]],
  profile: this.fb.group({
    firstName: ['', Validators.required],
    lastName: ['', Validators.required],
  })
});

// Subscribe to changes
this.form.valueChanges.pipe(
  debounceTime(300),
  distinctUntilChanged()
```

```
).subscribe(val => this.saveDraft(val));
```

Custom Async Validator

```
function usernameAvailable(api: ApiService): AsyncValidatorFn {  
  return (control: AbstractControl) => {  
    return timer(400).pipe(  
      switchMap(() => api.checkUsername(control.value)),  
      map(available => available ? null : { usernameTaken: true }),  
      catchError(() => of(null))  
    );  
  };  
}
```

■ For enterprise Angular applications, always prefer Reactive Forms. They are easier to test, support complex dynamic form scenarios, and integrate naturally with RxJS.

■ CROSS / FOLLOW-UP QUESTIONS

- What is FormArray? When do you use it?
- How do you implement cross-field validation (e.g., password === confirmPassword)?
- What is patchValue vs setValue in reactive forms?
- How do you reset a form while keeping validators?
- How do you lazily add controls to a FormGroup?
- What is ControlValueAccessor and how do you create a custom form control?

SECTION 17 · JWT AUTHENTICATION

Q19. How do you implement JWT authentication in Angular? Cover token storage, interceptor, and route protection.

■ ANSWER

JWT Structure

A JWT has three base64url-encoded parts separated by dots: Header.Payload.Signature. The payload contains claims (userId, role, exp). NEVER store sensitive data in JWT — it is only signed, not encrypted.

Auth Service

```
@Injectable({ providedIn: 'root' })  
export class AuthService {  
  private tokenKey = 'access_token';  
  private currentUser$ = new BehaviorSubject<User | null>(null);  
  readonly user$ = this.currentUser$.asObservable();
```

```

login(email: string, password: string): Observable<void> {
  return this.http.post<{ token: string }>('/api/auth/login',
    { email, password }).pipe(
    tap(res => {
      // Prefer sessionStorage or httpOnly cookie over localStorage
      sessionStorage.setItem(this.tokenKey, res.token);
      this.currentUser$.next(this.decodeToken(res.token));
    }),
    map(() => void 0)
  );
}

getToken(): string | null {
  return sessionStorage.getItem(this.tokenKey);
}

isAuthenticated(): boolean {
  const token = this.getToken();
  if (!token) return false;
  const payload = JSON.parse(atob(token.split('.')[1]));
  return payload.exp * 1000 > Date.now();
}

logout(): void {
  sessionStorage.removeItem(this.tokenKey);
  this.currentUser$.next(null);
}
}

```

JWT Interceptor

```

export const jwtInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).getToken();
  if (token) {
    req = req.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    });
  }
}

```

```
return next(req);  
};  
  
// Register in app.config.ts  
provideHttpClient(withInterceptors([jwtInterceptor]))
```

■ Token storage best practice: httpOnly cookies are safest (immune to XSS). If using localStorage/sessionStorage, ensure you have strong CSP headers and sanitise all user input to prevent XSS attacks.

■ CROSS / FOLLOW-UP QUESTIONS

- localStorage vs sessionStorage vs httpOnly cookie for JWT — which is most secure and why?
- How do you handle JWT token refresh (sliding sessions)?
- What is the difference between Authentication and Authorisation?
- How would you handle token expiry silently in the background?
- How do you protect against CSRF attacks in an Angular JWT implementation?

SECTION 18 · ROUTE GUARDS

Q20. Explain all types of Angular Route Guards with implementations.

■ ANSWER

Guard Types

- **canActivate** — Can the user access this route?
- **canActivateChild** — Can the user access child routes?
- **canDeactivate** — Can the user leave the current route (unsaved changes)?
- **canMatch** — (Angular 14+) Should this route definition even be matched?
- **resolve** — Pre-fetch data before activating a route.

Functional Guard (Angular 14+ — recommended)

```
// auth.guard.ts  
  
export const authGuard: CanActivateFn = (route, state) => {  
  const auth = inject(AuthService);  
  const router = inject(Router);  
  
  if (auth.isAuthenticated()) {  
    return true;  
  }  
  
  // Redirect to login, preserve intended URL
```

```

return router.createUrlTree(['/login'], {
  queryParams: { returnUrl: state.url }
});
};

// role.guard.ts
export const roleGuard = (requiredRole: string): CanActivateFn =>
(route, state) => {
  const user = inject(AuthService).currentUser;
  return user?.role === requiredRole
    ? true
    : inject(Router).createUrlTree(['/forbidden']);
};

// Usage in routes
{
  path: 'admin',
  canActivate: [authGuard, roleGuard('admin')],
  loadChildren: () => import('./admin/admin.routes')
}

```

canDeactivate — Unsaved Changes Guard

```

export interface CanDeactivateComponent {
  canDeactivate(): boolean | Observable<boolean>;
}

export const unsavedChangesGuard: CanDeactivateFn<CanDeactivateComponent> =
(component) => {
  if (component.canDeactivate()) return true;
  return confirm('You have unsaved changes. Leave anyway?');
};

```

■ In Angular 15+, class-based guards implementing `CanActivate` are deprecated. Use functional guards (`CanActivateFn`) instead — they are simpler, support `inject()`, and don't need a class.

■ CROSS / FOLLOW-UP QUESTIONS

- What is the difference between `canActivate` and `canMatch`?
- How do you pass data to a guard without using the route data property?
- How do you implement a guard that checks permissions from an async API call?
- What is the execution order when you have multiple guards on the same route?
- How does `canDeactivate` work with a confirm dialog from a UI library?

SECTION 19 · HTTP INTERCEPTOR

Q21. How do HTTP Interceptors work in Angular? Implement a retry interceptor with exponential backoff.

■ ANSWER

HTTP Interceptors sit in the request/response pipeline. Every outgoing request and incoming response passes through them in order. They are perfect for cross-cutting concerns: auth, logging, caching, retry, loading indicators.

Interceptor Pipeline

Request: Component → Interceptor 1 → Interceptor 2 → HttpBackend (server). Response: HttpBackend → Interceptor 2 → Interceptor 1 → Component.

```
// retry.interceptor.ts

export const retryInterceptor: HttpInterceptorFn = (req, next) => {
  const MAX_RETRIES = 3;
  const RETRY_DELAY = 1000; // ms base delay

  return next(req).pipe(
    retryWhen(errors =>
      errors.pipe(
        // Only retry on network errors or 5xx
        mergeMap((err: HttpErrorResponse, attempt) => {
          if (attempt >= MAX_RETRIES || err.status < 500) {
            return throwError(() => err);
          }
          const delay = RETRY_DELAY * Math.pow(2, attempt); // exponential
          console.warn(`Retry ${attempt + 1} in ${delay}ms`);
          return timer(delay);
        })
      )
    );
};

// loading.interceptor.ts
```



```

export const loadingInterceptor: HttpInterceptorFn = (req, next) => {
  const loading = inject>LoadingService);
  loading.show();

  return next(req).pipe(
    finalize(() => loading.hide())
  );
};

// Register all interceptors
provideHttpClient(
  withInterceptors([
    jwtInterceptor,
    retryInterceptor,
    loadingInterceptor,
  ])
)

```

■ The ORDER of interceptors matters. In `withInterceptors([A, B, C])`, requests flow `A→B→C→backend` and responses flow `C→B→A`. Put auth interceptor first so others can see auth headers.

■ CROSS / FOLLOW-UP QUESTIONS

- How do you skip an interceptor for specific requests (e.g., refresh token call)?
- What is `HttpContext` and how do you use it to pass metadata to an interceptor?
- How do you implement a caching interceptor?
- What is the difference between functional interceptors and class-based interceptors?
- How do you test an HTTP interceptor?

SECTION 20 · SIGNALS IN ANGULAR (ANGULAR 16+)

Q22. What are Angular Signals? How do they differ from RxJS Observables? When do you use each?

■ ANSWER

Signals are Angular's reactive primitive introduced in v16. A signal is a wrapper around a value that notifies consumers when that value changes. Unlike Observables, signals are synchronous, always hold a current value, and don't require `subscribe/unsubscribe`.

Signal API

```

import { signal, computed, effect } from '@angular/core';

// 1. signal() – writable reactive value
const count = signal(0);
count(); // read: 0
count.set(5); // write: 5
count.update(v => v + 1); // update: 6

// 2. computed() – derived value, lazy & memoised
const doubled = computed(() => count() * 2);
const isEven = computed(() => count() % 2 === 0);

// 3. effect() – side effects when signals change
effect(() => {
  console.log('Count changed to:', count());
  // Automatically tracks any signals read inside
});

// Component example
@Component({
  template: `
    <p>Count: {{ count() }}</p>
    <p>Doubled: {{ doubled() }}</p>
    <button (click)="increment()">+1</button>
  `
})
export class CounterComponent {
  count = signal(0);
  doubled = computed(() => this.count() * 2);

  increment() { this.count.update(v => v + 1); }
}

```

Signal Inputs (Angular 17+)

```

// Instead of @Input() decorator
@Component({ ... })
export class CardComponent {
  title = input<string>(''); // optional with default
}

```

```

userId = input.required<number>(); // required, no default
onClose = output<void>(); // replaces @Output()
}

// Usage
<app-card [title]="Hello" [userId]="42" (onClose)="handleClose()">

```

Signals vs Observables

- **Signals:** Synchronous, always have a value, no subscription needed. Best for component state.
- **Observables:** Async, event streams, operators (map/filter/switchMap). Best for HTTP, WebSocket, timers.
- Use **toSignal(obs\$)** to convert Observable → Signal.
- Use **toObservable(sig)** to convert Signal → Observable.

■ Signals enable fine-grained reactivity. Angular's template engine can update **ONLY** the parts of the DOM that depend on a changed signal — without running full change detection. This is the basis of Angular's zoneless mode.

■ CROSS / FOLLOW-UP QUESTIONS

- What is the difference between signal() and BehaviorSubject?
- How do signals work with change detection — especially OnPush and zoneless?
- What are the limitations of effect() — can you update a signal inside an effect?
- How do you use toSignal() with an HTTP Observable?
- What is linkedSignal() introduced in Angular 19?
- How do signals enable zoneless Angular (no zone.js)?

SECTION 21 · ANGULAR 20+ & STANDALONE ARCHITECTURE

Q23. What is Standalone Architecture in Angular? Explain the migration from NgModule-based to standalone.

■ ANSWER

Standalone components, directives, and pipes do not belong to any NgModule. They manage their own dependencies directly in the @Component, @Directive, or @Pipe decorator's imports array. This was introduced in Angular 14 and became the default in Angular 17.

Standalone Component

```

// Before (module-based)

@NgModule({
  declarations: [UserCardComponent],

```

```

imports: [CommonModule, RouterModule],
})
export class UserModule {}

// After (standalone)
@Component({
  selector: 'app-user-card',
  standalone: true, // ← key flag (default in Angular 17+)
  imports: [CommonModule, RouterLink], // ← declare dependencies here
  template: `
<div>{{ user.name }}</div>
<a [routerLink]="['/user', user.id]">View Profile</a>
`
})
export class UserCardComponent {
  @Input() user!: User;
}

```

app.config.ts — Replaces AppModule Providers

```

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes, withPreloading(PreloadAllModules)),
    provideHttpClient(withInterceptors([authInterceptor])),
    provideAnimations(),
    provideStore(reducers), // NgRx
    importProvidersFrom(SomeModule), // bridge for legacy modules
  ]
};

```

Angular 20+ Key Changes

- Standalone is the default — no --standalone flag needed in CLI.
- NgModules still work but new projects start module-free.
- Signal-based components are becoming the standard (input(), output(), model()).
- Deferred loading (@defer) for template sections.
- Built-in control flow (@if, @for, @switch) replacing structural directives.
- Zoneless change detection via provideExperimentalZonelessChangeDetection().

```

<!-- Angular 17+ built-in control flow -->
@if (isLoggedIn) {
  <app-dashboard />
} @else {

```

```

<app-login />
}

@for (item of items; track item.id) {
<app-item [data]="item" />
} @empty {
<p>No items found</p>
}

@defer (on viewport) {
<app-heavy-chart />
} @placeholder {
<div>Loading chart...</div>
} @loading (minimum 500ms) {
<app-spinner />
}

```

■ @defer is a game-changer for performance — it lazy-loads components at the template level based on triggers: viewport, idle, interaction, timer, when(condition).

■ CROSS / FOLLOW-UP QUESTIONS

- How do you migrate an existing NgModule-based app to standalone?
- What is importProvidersFrom() and why is it a bridge pattern?
- How does @defer differ from lazy-loaded routes?
- What are the differences between @if and *ngIf at the compiler level?
- What is the Angular Signals-based component model roadmap?

SECTION 22 · ■■ MICRO-FRONTEND ARCHITECTURE (BONUS — ARCHITECTURE LEVEL)

Q24. What is Micro-Frontend architecture? How do you implement it in Angular using Module Federation?

■ ANSWER

Micro-Frontend (MFE) is an architectural pattern where a frontend application is decomposed into smaller, independently deployable applications (remotes), each owned by a separate team, unified by a shell/host application at runtime.

Core Principles

- **Independent deployment** — Each MFE can be built, tested, and deployed without affecting others.
- **Technology agnostic (optionally)** — Different MFES could use different frameworks (though same is preferred for sharing).
- **Isolated team ownership** — Each team owns its domain end-to-end.
- **Shared nothing by default** — Shared libraries are carefully managed to avoid version conflicts.

Webpack Module Federation — Configuration

```
// shell (host) webpack.config.js

const { ModuleFederationPlugin } = require('@module-federation/webpack');

module.exports = {
  plugins: [
    new ModuleFederationPlugin({
      name: 'shell',
      remotes: {
        // runtime URL – can come from a config server
        usersMfe: 'usersMfe@http://localhost:4201/remoteEntry.js',
        ordersMfe: 'ordersMfe@http://localhost:4202/remoteEntry.js',
      },
      shared: {
        '@angular/core': { singleton: true, strictVersion: true },
        '@angular/router': { singleton: true, strictVersion: true },
        '@angular/common': { singleton: true, strictVersion: true },
      }
    })
  ]
};

// users-mfe webpack.config.js (remote)

module.exports = {
  plugins: [
    new ModuleFederationPlugin({
      name: 'usersMfe',
      filename: 'remoteEntry.js',
      exposes: {
        './UsersModule': './src/app/users/users.module.ts',
        // OR standalone routes:
        './routes': './src/app/users/users.routes.ts',
      },
    })
  ]
};
```

```

shared: { /* same shared config */ }
})
]
};

```

Dynamic Remote Loading in Shell Router

```

// shell app.routes.ts
{
  path: 'users',
  loadChildren: () =>
  loadRemoteModule({
    type: 'module',
    remoteEntry: 'http://localhost:4201/remoteEntry.js',
    exposedModule: './routes',
  }).then(m => m.usersRoutes),
  canActivate: [authGuard],
},

```

Communication Between MFES

- **Shared state library** — A shared npm package exporting a RxJS Subject or Signal store.
- **Custom Events (`window.dispatchEvent`)** — Loose coupling via browser native events.
- **URL / query params** — Routing-based communication.
- **Session Storage** — Simple key-value for auth tokens, user context.
- **Event Bus service** — Singleton in the shared zone.

```

// shared-event-bus.ts (published as @company/shared-lib)
@Injectable({ providedIn: 'root' })
export class EventBusService {
  private eventMap = new Map<string, Subject<any>>();

  emit<T>(event: string, data: T) {
    if (!this.eventMap.has(event)) {
      this.eventMap.set(event, new Subject());
    }
    this.eventMap.get(event)!.next(data);
  }

  on<T>(event: string): Observable<T> {
    if (!this.eventMap.has(event)) {
      this.eventMap.set(event, new Subject());
    }
  }
}

```

```

return this.eventMap.get(event)!.asObservable();
}
}

// In users-mfe
this.eventBus.emit('user:selected', { id: 42 });

// In orders-mfe
this.eventBus.on<{ id: number }>('user:selected')
  .subscribe(user => this.loadOrders(user.id));

```

Challenges & Solutions

- **Version conflicts:** Use `singleton:true + strictVersion` in Module Federation shared config.
- **Routing:** Shell owns top-level routes; each MFE owns its child routes. Use `activatedRoute` carefully.
- **Authentication:** Shell handles auth; passes token via shared service or storage.
- **CSS isolation:** Use Shadow DOM or BEM naming conventions to prevent style leaks.
- **Testing:** Each MFE has its own test suite. Integration testing uses Cypress with all MFEs running.
- **Deployment:** Each MFE deployed to its own CDN path. Shell reads runtime config from a manifest file.

■ `@angular-architects/module-federation` is the canonical Angular library. In 2024+, Module Federation 2.0 (from ByteDance) offers improved TypeScript support and dynamic federation without Webpack config files.

■ CROSS / FOLLOW-UP QUESTIONS

- How do you handle global state (like auth) across multiple MFEs?
- What happens when two MFEs load different versions of `@angular/core`?
- How do you implement shell-level error boundaries for a failing remote MFE?
- How do you test a micro-frontend in isolation vs in the shell?
- What are the trade-offs of MFE vs a well-structured monorepo (Nx)?
- How do you implement a graceful fallback if a remote MFE is unavailable?
- What is Native Federation and how does it differ from Webpack Module Federation?
- How does CI/CD change in a micro-frontend architecture — explain the deployment pipeline.

SECTION 23 · PERFORMANCE OPTIMIZATION & BEST PRACTICES

Q25. What are the key techniques for optimising Angular application performance?

■ ANSWER

Build-time Optimisations

- **Ahead-of-Time (AOT) compilation** — Default since Angular 9. Templates compiled at build time, not at runtime.
- **Tree-shaking** — Unused code is removed by the bundler (Webpack/esbuild).
- **Code splitting** — Lazy-loaded routes create separate chunks.
- **esbuild / Vite** — Angular 17+ uses esbuild by default, dramatically faster builds.

Runtime Optimisations

- **OnPush change detection** — Components only check when inputs change or events fire.
- **trackBy in *ngFor / @for** — Prevents unnecessary DOM re-creation.
- **async pipe** — Automatically manages subscriptions with OnPush.
- **Signals** — Fine-grained reactivity — only affected DOM nodes update.
- **Virtual scrolling** — CDK's CdkVirtualScrollViewport for large lists.
- **@defer** — Lazy-load heavy components at template level.
- **Preloading strategy** — QuicklinkStrategy or custom strategy to preload likely routes.

```
// trackBy prevents full list re-render
@for (item of items; track item.id) {
<app-item [data]="item" />
}

// Old syntax
<li *ngFor="let item of items; trackBy: trackById">
trackById = (index: number, item: Item) => item.id;

// Virtual scrolling for large lists
<cdk-virtual-scroll-viewport itemSize="48" style="height: 400px">
<div *cdkVirtualFor="let item of hugeList">{{ item.name }}</div>
</cdk-virtual-scroll-viewport>
```

Bundle Optimisation

- Analyse bundles with: `ng build --stats-json` → `webpack-bundle-analyzer`.
- Avoid importing entire libraries — use named imports.
- Use `moment` → `date-fns` or `luxon` (smaller footprint).
- Defer third-party scripts that aren't needed at startup.

■ The biggest gains usually come from: 1) OnPush everywhere, 2) lazy loading all routes, 3) async pipe, 4) trackBy in lists. Signals + zoneless mode is the future for maximum performance.

■ CROSS / FOLLOW-UP QUESTIONS

- What is the difference between JIT and AOT compilation?
- How does esbuild improve Angular build times vs Webpack?

- How do you measure Angular app performance? What tools do you use?
- What is the Largest Contentful Paint (LCP) and how does Angular SSR improve it?
- How does Angular's hydration (SSR) work in Angular 16+?
- What is deferrable views (@defer) and how do you configure its triggers?

Q26. Scenario: You have an e-commerce app with 500ms LCP on mobile. Walk through your performance investigation and fixes.

■ ANSWER

Step 1 — Measure

- Run Lighthouse in Chrome DevTools on the production build.
- Use Angular DevTools extension to inspect component tree and change detection cycles.
- Check Network tab: bundle sizes, number of requests, waterfall.
- Use Coverage tab to find unused JS/CSS.

Step 2 — Identify Issues

- Large initial bundle (check: `ng build --stats-json + webpack-bundle-analyzer`).
- Eagerly-loaded routes/modules that should be lazy.
- Images not using `NgOptimizedImage` (no lazy loading, no `srcset`).
- Missing `OnPush` on large component trees.
- No SSR — first paint blocked by client-side rendering.

Step 3 — Fixes

```
// 1. Lazy load all feature routes
{
  path: 'products',
  loadChildren: () =>
    import('./products/product-list.component')
      .then(m => m.ProductListComponent),
}

// 2. NgOptimizedImage for hero images
<img ngSrc="/hero.jpg" width="1200" height="600"
  priority fetchpriority="high">

// 3. SSR + Hydration (Angular 16+)
// app.config.server.ts
export const config: ApplicationConfig = {
  providers: [provideServerRendering()]
};
```

```
// 4. @defer heavy sections
@defer (on idle) {
  <app-recommendations />
} @placeholder {
  <div class="placeholder-box"></div>
}

// 5. Preconnect to API domain in index.html
<link rel="preconnect" href="https://api.example.com">
```

■ On mobile: every KB matters. Target < 200KB initial JS (gzipped). Use Chrome's Slow 3G throttling and CPU 4x slowdown to simulate real-world mobile performance.

■ CROSS / FOLLOW-UP QUESTIONS

- How does Angular SSR (Angular Universal) work and what does partial hydration mean?
- What is the Interaction to Next Paint (INP) metric and how does Angular affect it?
- How does NgOptimizedImage improve Core Web Vitals?
- When would you use a Service Worker (PWA) in an Angular app?

SECTION 24 · TESTING ANGULAR APPLICATIONS

Q27. How do you test Angular components, services, and interceptors?
Show unit test examples.

■ ANSWER

Testing a Service

```
describe('UserService', () => {
  let service: UserService;
  let httpMock: HttpTestingController;

  beforeEach(() => {
    TestBed.configureTestingModule({
      imports: [HttpClientTestingModule],
      providers: [UserService],
    });
    service = TestBed.inject(UserService);
    httpMock = TestBed.inject(HttpTestingController);
```

```

});

it('should fetch a user by ID', () => {
  const mockUser: User = { id: 1, name: 'Alice' };

  service.getUser(1).subscribe(user => {
    expect(user).toEqual(mockUser);
  });

  const req = httpMock.expectOne('/api/users/1');
  expect(req.request.method).toBe('GET');
  req.flush(mockUser);
});

afterEach(() => httpMock.verify());
});

```

Testing a Component

```

describe('UserCardComponent', () => {
  let fixture: ComponentFixture<UserCardComponent>;
  let component: UserCardComponent;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [UserCardComponent], // standalone component
    }).compileComponents();

    fixture = TestBed.createComponent(UserCardComponent);
    component = fixture.componentInstance;
    component.user = { id: 1, name: 'Bob', email: 'bob@test.com' };
    fixture.detectChanges();
  });

  it('should display user name', () => {
    const el = fixture.nativeElement.querySelector('h2');
    expect(el.textContent).toContain('Bob');
  });
});

```

```
it('should emit selected event on click', () => {
  spyOn(component.selected, 'emit');
  fixture.nativeElement.querySelector('button').click();
  expect(component.selected.emit).toHaveBeenCalledWith(component.user);
});
});
```

■ Use TestBed.overrideComponent() to replace child components with stubs, keeping tests fast and isolated. Use NO_ERRORS_SCHEMA only as a last resort.

■ CROSS / FOLLOW-UP QUESTIONS

- What is the difference between shallow and deep rendering in Angular tests?
- How do you test a component with router-link without setting up RouterTestingModule?
- How do you test an HTTP interceptor?
- What is the harness pattern in Angular CDK testing?
- How do you test Signals and computed values?

SECTION 25 · ADVANCED SCENARIO-BASED QUESTIONS

Q28. Scenario: Build a real-time dashboard with live stock prices using WebSocket in Angular.

■ ANSWER

This tests your ability to combine services, RxJS, OnPush, and cleanup patterns in a production scenario.

WebSocket Service

```
@Injectable({ providedIn: 'root' })
export class StockWebSocketService {
  private socket$: WebSocketSubject<StockUpdate>;

  connect(symbols: string[]): Observable<StockUpdate> {
    this.socket$ = websocket({
      url: 'wss://market.api.com/ws',
      openObserver: {
        next: () => {
          this.socket$.next({ action: 'subscribe', symbols });
        }
      }
    });
  }
}
```

```

});

return this.socket$.pipe(
  retryWhen(errors =>
    errors.pipe(
      tap(err => console.error('WS error, reconnecting...', err)),
      delayWhen(() => timer(3000))
    )
  ),
  share()
);
}

disconnect() {
  this.socket$?.complete();
}
}

```

Dashboard Component

```

@Component({
  selector: 'app-stock-dashboard',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    @for (stock of stocks$ | async; track stock.symbol) {
      <app-stock-ticker
        [data]="stock"
        [class.up]="stock.change > 0"
        [class.down]="stock.change < 0">
      </app-stock-ticker>
    }
  `
})
export class StockDashboardComponent implements OnDestroy {
  private destroyed$ = new Subject<void>();

  stocks$ = this.wsService.connect(['AAPL', 'GOOGL', 'MSFT']).pipe(
    scan((acc, update) => {

```

```

const idx = acc.findIndex(s => s.symbol === update.symbol);
if (idx > -1) {
  const updated = [...acc];
  updated[idx] = { ...updated[idx], ...update };
  return updated;
}
return [...acc, update];
}, [] as StockUpdate[]),
takeUntil(this.destroyed$)
);

constructor(private wsService: StockWebSocketService) {}

ngOnDestroy() {
  this.destroyed$.next();
  this.destroyed$.complete();
  this.wsService.disconnect();
}
}

```

■ The `scan()` operator accumulates state from a stream of updates — like a reducer. Combined with `takeUntil(destroyed$)` and `OnPush`, this is a production-ready pattern.

■ CROSS / FOLLOW-UP QUESTIONS

- How do you handle WebSocket reconnection with exponential backoff?
- How would you throttle very high-frequency WebSocket updates (100/sec) to not overwhelm the UI?
- How does this component behave with `ChangeDetectionStrategy.OnPush`? Why is it correct?
- How would you add a filter to only show selected symbols?
- How would you test this component?

Q29. Scenario: Design a scalable state management solution for a large Angular e-commerce app without using NgRx.

■ ANSWER

This tests architectural thinking — the ability to design scalable patterns without external libraries.

Signal-based Store Pattern

```

// cart.store.ts – Feature store
@Injectable({ providedIn: 'root' })
export class CartStore {
  // Private writable state

```

```

private _items = signal<CartItem[]>([]);
private _loading = signal(false);
private _error = signal<string | null>(null);

// Public read-only signals
readonly items = this._items.asReadOnly();
readonly loading = this._loading.asReadOnly();
readonly error = this._error.asReadOnly();

// Derived computed signals
readonly itemCount = computed(() => this._items().length);
readonly totalPrice = computed(() =>
this._items().reduce((sum, item) => sum + item.price * item.qty, 0)
);
readonly isEmpty = computed(() => this._items().length === 0);

constructor(private api: CartApiService) {}

async addItem(product: Product, qty = 1) {
this._loading.set(true);
this._error.set(null);
try {
const updated = await firstValueFrom(
this.api.addToCart(product.id, qty)
);
this._items.set(updated);
} catch (e: any) {
this._error.set(e.message);
} finally {
this._loading.set(false);
}
}

removeItem(itemId: number) {
this._items.update(items => items.filter(i => i.id !== itemId));
}

updateQty(itemId: number, qty: number) {

```



```
this._items.update(items =>
items.map(i => i.id === itemId ? { ...i, qty } : i)
);
}
}

// In component – no subscribe needed
readonly cart = inject(CartStore);

// template
{{ cart.itemCount() }} items – {{ cart.totalPrice() | currency }}
```

■ This pattern gives you: type safety, computed derivations, no boilerplate, and fine-grained reactivity. It scales well for medium apps. For large teams, NgRx or NGXS add time-travel debugging and stricter action/reducer separation.

■ CROSS / FOLLOW-UP QUESTIONS

- How does this compare to NgRx in terms of boilerplate and scalability?
 - How would you persist this state to localStorage?
 - How would you implement optimistic updates with error rollback in this store?
 - How do you share state between lazy-loaded feature modules?
 - When would you choose NgRx over a custom signal store?
-

End of Angular Deep-Dive Interview Questions

Good luck with your interviews! ■

Key topics covered: Components · Lifecycle Hooks · Data Binding · Directives · Pipes · DI · Routing · RxJS · Observables · HttpClient · Interceptors · Forms · JWT · Route Guards · Signals · Standalone · Angular 20+ · Micro-Frontend · Performance · Testing